МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

ФАКУЛЬТЕТ АВТОМАТИКИ И ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ
Кафедра автоматизированных систем управления

ОТЧЕТ

по дисциплине Современные технологии разработки программного обеспечения

ЛАБОРАТОРНАЯ РАБОТА № 7

Тема: Создание приложений с меню

Выполнил:
Епифанцев Г.А.
Группа АПМ-25

Подпись: _____ Дата: ___

Проверил:
Эстрайх И.В.

Оценка _____

Подпись: _____ Дата: ___

Новосибирск 2026

СОДЕРЖАНИЕ

Постановка задачи **3**

1 Ход работы **4**

1.1 Главный экран приложений 4

1.2 Обычное меню активности 5

1.3 Обработка выбора в обычном меню 6

1.4 Всплывающее меню из XML 7

1.5 Группы, подменю и отметка 8

1.6 Программное добавление пунктов 9

Заключение **10**

Приложение А. Исходный код **11**

ПОСТАНОВКА ЗАДАЧИ

Цель работы

Освоить создание обычного меню активности и всплывающего меню PopupMenu в Android-приложениях на Java и Kotlin.

Задание

1. Создать приложения на Java и Kotlin, в которых используется обычное и всплывающее меню.
2. Включить в отчёт цель работы, код программ, скриншоты результатов и вывод.

1 Ход работы

1.1 Главный экран приложений

Созданы два самостоятельных проекта Android Studio. Lab7MenusJava содержит приложение на Java, а Lab7MenusKotlin содержит приложение на Kotlin. В обоих проектах разметка главного экрана описана в файле activity_main.xml.

Верхняя панель Toolbar подключена как панель действий через setSupportActionBar(). Значок с тремя точками открывает обычное меню. На главном экране находятся кнопка, текст и изображение Swift для открытия PopupMenu, отдельная кнопка для программного меню и два поля для отображения выбора и состояния меню.

Главные экраны приложений показаны на рисунках 1 и 2.

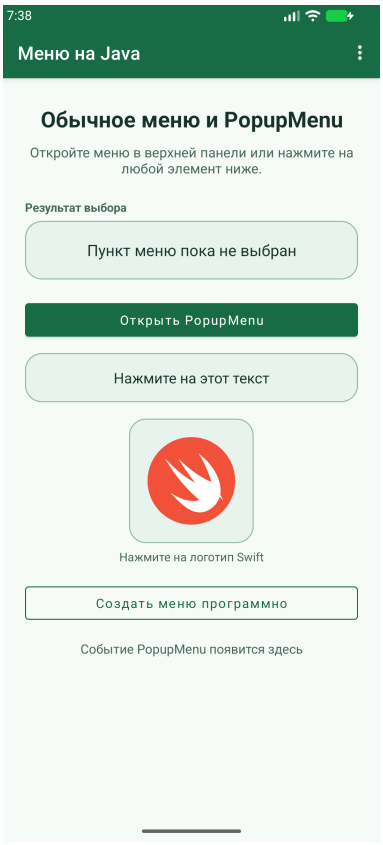


Рисунок 1 — Главный экран Java-приложения

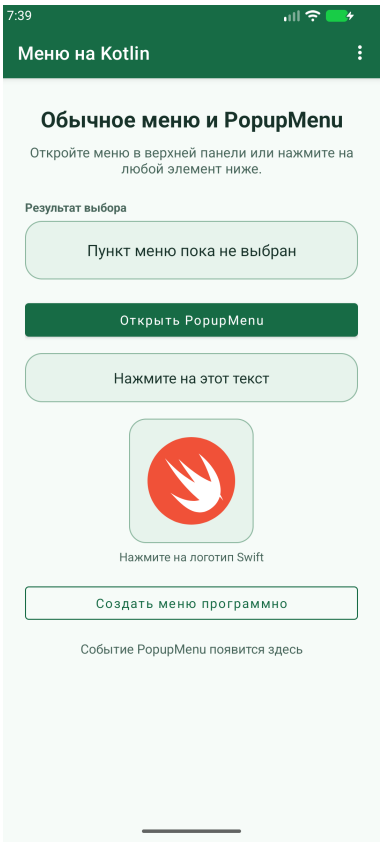


Рисунок 2 — Главный экран Kotlin-приложения

1.2 Обычное меню активности

Ресурс res/menu/menu_main.xml описывает пункт Settings и группу языков Java, Kotlin и Python. Атрибут checkableBehavior со значением single задаёт одиночный выбор. При первом открытии отмечен Kotlin. Атрибут showAsAction со значением never помещает пункты в меню верхней панели.

В обеих версиях метод onCreateOptionsMenu() подключает ресурс через MenuInflater. Ниже приведён соответствующий метод на Java.

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    getMenuInflater().inflate(R.menu.menu_main, menu);
    return true;
}
```

Обычное меню в приложениях на Java и Kotlin показано на рисунках 3 и 4.

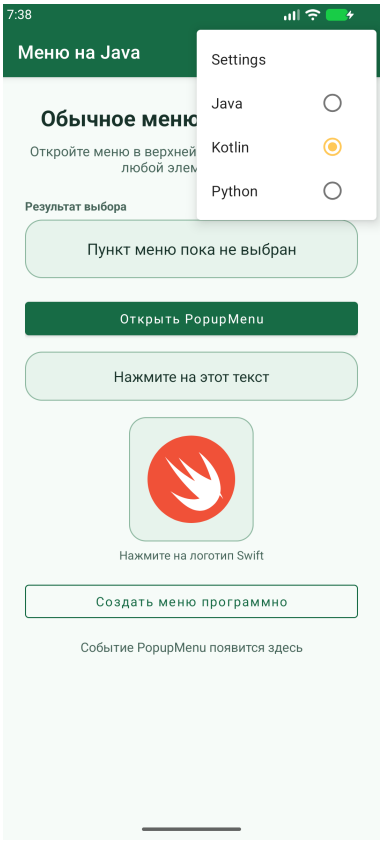


Рисунок 3 — Обычное меню на Java

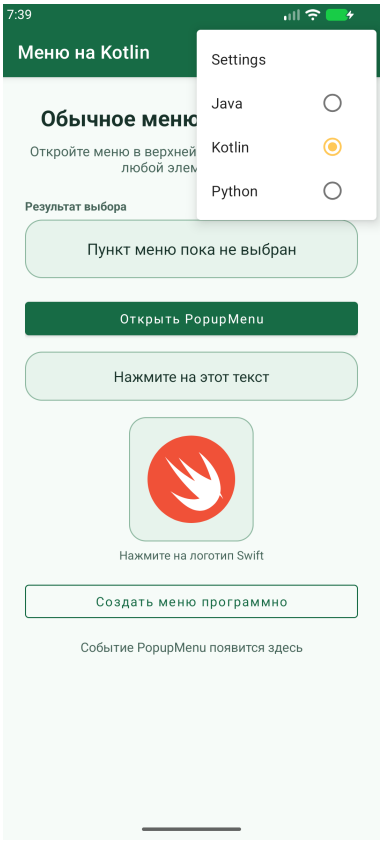


Рисунок 4 — Обычное меню на Kotlin

1.3 Обработка выбора в обычном меню

Метод onOptionsItemSelected() определяет выбранный пункт по идентификатору. При выборе Settings в поле результата появляется название пункта. При выборе языка метод отмечает соответствующий пункт и выводит его название. Kotlin-версия использует для той же обработки выражение when.

```
@Override
public boolean onOptionsItemSelected(MenuItem item) {
    int id = item.getItemId();
    if (id == R.id.action_settings) {
        resultText.setText(R.string.selected_settings);
        return true;
    }
    if (id == R.id.action_java || id == R.id.action_kotlin
            || id == R.id.action_python) {
        item.setChecked(true);
        resultText.setText(
                getString(R.string.selected_language, item.getTitle()));
        return true;
    }
    return super.onOptionsItemSelected(item);
}
```

На рисунке 5 в поле результата отображается выбранный язык Java.

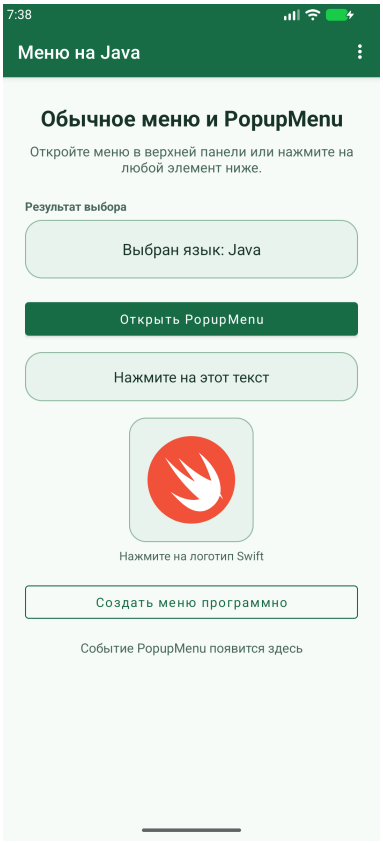


Рисунок 5 — Результат выбора языка

1.4 Всплывающее меню из XML

Меню загружается из ресурса popup_menu.xml. Кнопка, текстовая область и изображение Swift вызывают один обработчик. Он получает нажатый элемент и привязывает к нему меню. При открытии и закрытии обновляется отдельное поле состояния.

```
private fun showPopupMenu(anchor: View) {
    val popupMenu = PopupMenu(this, anchor)
    popupMenu.inflate(R.menu.popup_menu)
    popupMenu.menu.findItem(R.id.popup_checked_item)
        .isChecked = popupItemChecked
    popupMenu.setOnMenuItemClickListener(::handlePopupItem)
    popupMenu.setOnDismissListener {
        eventText.setText(R.string.popup_closed)
    }
    popupMenu.show()
    eventText.setText(R.string.popup_opened)
}
```

На рисунке 6 меню открыто нажатием на кнопку, на рисунке 7 нажатием на изображение Swift.

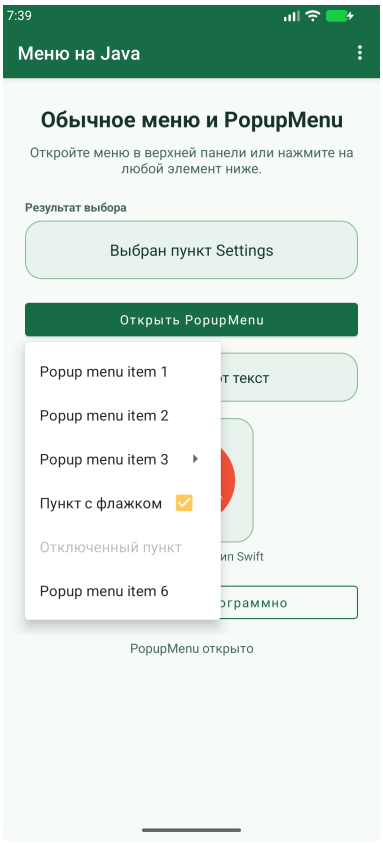


Рисунок 6 — PopupMenu у кнопки

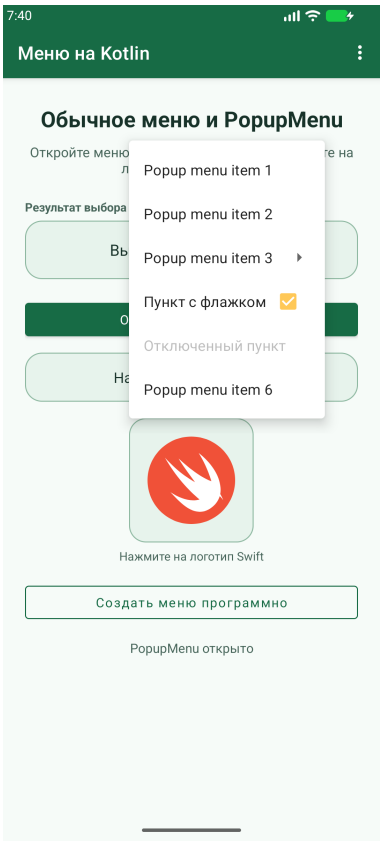


Рисунок 7 — PopupMenu у изображения Swift

1.5 Группы, подменю и отметка

В ресурсе всплывающего меню две группы. В первой находятся обычные пункты и пункт с вложенным меню. Во второй находятся пункт с флажком, отключённый пункт и обычный пункт. При первом открытии флажок установлен. Нажатие переключает его, а при повторном открытии видна последняя выбранная отметка. Отключённый пункт отображается серым.

Нажатие на Popup menu item 3 открывает вложенный список, показанный на рисунке 8. При переходе к нему родительское PopupMenu закрывается, и setOnDismissListener() обновляет строку состояния. После выбора вложенного пункта его название появляется в поле результата, как показано на рисунке 9.

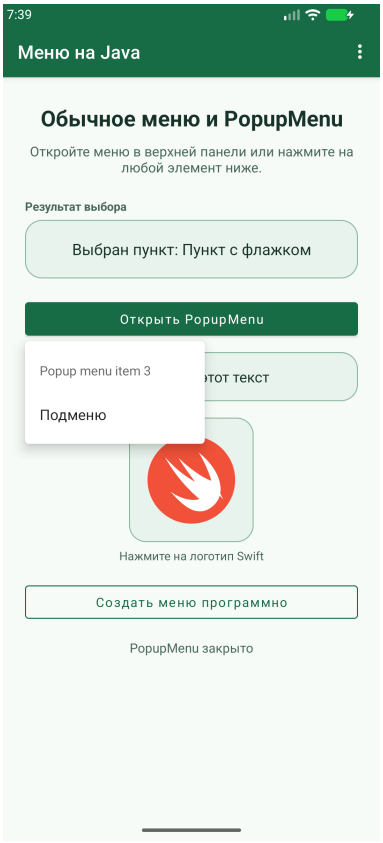


Рисунок 8 — Вложенное меню

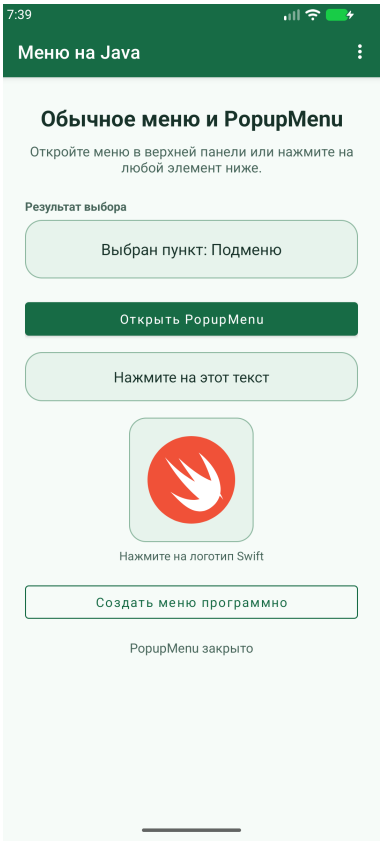


Рисунок 9 — Выбор пункта и закрытие PopupMenu

1.6 Программное добавление пунктов

Для второго PopupMenu пункты slot1 и slot2 добавляются методом Menu.add(). Идентификатор группы, идентификаторы пунктов и заголовки берутся из ресурсов. Обработчик нажатия выводит название выбранного пункта.

```
popupMenu.getMenu().add(
        R.id.dynamic_group, R.id.dynamic_first, 1, R.string.dynamic_first);
popupMenu.getMenu().add(
        R.id.dynamic_group, R.id.dynamic_second, 2, R.string.dynamic_second
    );
popupMenu.setOnMenuItemClickListener(item -> {
    resultText.setText(
            getString(R.string.dynamic_selected, item.getTitle()));
    return true;
});
```

На рисунке 10 показано меню с двумя программно добавленными пунктами.

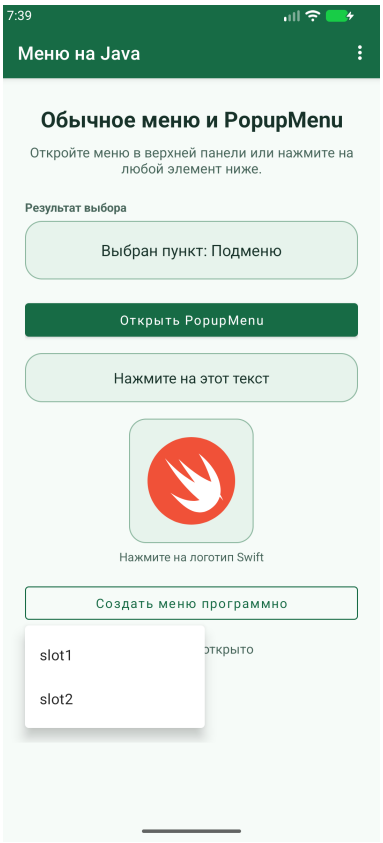


Рисунок 10 — Программно созданное меню

ЗАКЛЮЧЕНИЕ

В приложениях на Java и Kotlin реализованы обычное меню активности с одиночным выбором языка и всплывающее меню с группами и подменю. PopupMenu открывается от трёх элементов экрана, поддерживает переключаемый флажок и отключённый пункт. Результат выбора и событие закрытия выводятся в отдельных полях. Для второго всплывающего меню пункты добавляются программно. Работа меню проверена в обоих приложениях на эмуляторе Android.

ПРИЛОЖЕНИЕ А

Исходный код

Java. MainActivity.java

```
package ru.nstu.lab7.javaapp;

import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;
import androidx.appcompat.widget.PopupMenu;
import androidx.appcompat.widget.Toolbar;

public class MainActivity extends AppCompatActivity {
    private TextView resultText;
    private TextView eventText;
    private boolean popupItemChecked = true;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        Toolbar toolbar = findViewById(R.id.toolbar);
        setSupportActionBar(toolbar);

        resultText = findViewById(R.id.resultText);
        eventText = findViewById(R.id.eventText);

        findViewById(R.id.popupButton).setOnClickListener(this::
   showPopupMenu);
        findViewById(R.id.popupText).setOnClickListener(this::showPopupMenu
   );
        findViewById(R.id.swiftImage).setOnClickListener(this::
   showPopupMenu);
        findViewById(R.id.dynamicButton).setOnClickListener(this::
   showProgrammaticMenu);
    }

    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }

    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
        int id = item.getItemId();
        if (id == R.id.action_settings) {
            resultText.setText(R.string.selected_settings);
            return true;
        }
        if (id == R.id.action_java || id == R.id.action_kotlin || id == R.
   id.action_python) {
            item.setChecked(true);
```

```
            resultText.setText(getString(R.string.selected_language, item.
    getTitle()));
            return true;
        }
        return super.onOptionsItemSelected(item);
    }

    private void showPopupMenu(View anchor) {
        PopupMenu popupMenu = new PopupMenu(this, anchor);
        popupMenu.inflate(R.menu.popup_menu);
        popupMenu.getMenu().findItem(R.id.popup_checked_item).setChecked(
    popupItemChecked);
        popupMenu.setOnMenuItemClickListener(this::handlePopupItem);
        popupMenu.setOnDismissListener(menu -> eventText.setText(R.string.
    popup_closed));
        popupMenu.show();
        eventText.setText(R.string.popup_opened);
    }

    private boolean handlePopupItem(MenuItem item) {
        int id = item.getItemId();
        if (id == R.id.popup_checked_item) {
            popupItemChecked = !item.isChecked();
        }
        if (id == R.id.popup_one
                || id == R.id.popup_two
                || id == R.id.popup_submenu_item
                || id == R.id.popup_checked_item
                || id == R.id.popup_six) {
            resultText.setText(getString(R.string.selected_popup, item.
    getTitle()));
            return true;
        }
        return false;
    }

    private void showProgrammaticMenu(View anchor) {
        PopupMenu popupMenu = new PopupMenu(this, anchor);
        popupMenu.getMenu().add(
                R.id.dynamic_group,
                R.id.dynamic_first,
                1,
                R.string.dynamic_first
        );
        popupMenu.getMenu().add(
                R.id.dynamic_group,
                R.id.dynamic_second,
                2,
                R.string.dynamic_second
        );
        popupMenu.setOnMenuItemClickListener(item -> {
            resultText.setText(getString(R.string.dynamic_selected, item.
    getTitle()));
            return true;
        });
        popupMenu.setOnDismissListener(menu -> eventText.setText(R.string.
    popup_closed));
        popupMenu.show();
        eventText.setText(R.string.popup_opened);
    }
}
```

Kotlin. MainActivity.kt

```
package ru.nstu.lab7.kotlinapp

import android.os.Bundle
import android.view.Menu
import android.view.MenuItem
import android.view.View
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import androidx.appcompat.widget.PopupMenu
import androidx.appcompat.widget.Toolbar

class MainActivity : AppCompatActivity() {
    private lateinit var resultText: TextView
    private lateinit var eventText: TextView
    private var popupItemChecked = true

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val toolbar: Toolbar = findViewById(R.id.toolbar)
        setSupportActionBar(toolbar)

        resultText = findViewById(R.id.resultText)
        eventText = findViewById(R.id.eventText)

        findViewById<View>(R.id.popupButton).setOnClickListener(::
   showPopupMenu)
        findViewById<View>(R.id.popupText).setOnClickListener(::
   showPopupMenu)
        findViewById<View>(R.id.swiftImage).setOnClickListener(::
   showPopupMenu)
        findViewById<View>(R.id.dynamicButton).setOnClickListener(::
   showProgrammaticMenu)
    }

    override fun onCreateOptionsMenu(menu: Menu): Boolean {
        menuInflater.inflate(R.menu.menu_main, menu)
        return true
    }

    override fun onOptionsItemSelected(item: MenuItem): Boolean = when (
   item.itemId) {
        R.id.action_settings -> {
            resultText.setText(R.string.selected_settings)
            true
        }

        R.id.action_java, R.id.action_kotlin, R.id.action_python -> {
            item.isChecked = true
            resultText.text = getString(R.string.selected_language, item.
   title)
            true
        }

        else -> super.onOptionsItemSelected(item)
    }
```

```
    private fun showPopupMenu(anchor: View) {
        val popupMenu = PopupMenu(this, anchor)
        popupMenu.inflate(R.menu.popup_menu)
        popupMenu.menu.findItem(R.id.popup_checked_item).isChecked =
    popupItemChecked
        popupMenu.setOnMenuItemClickListener(::handlePopupItem)
        popupMenu.setOnDismissListener {
            eventText.setText(R.string.popup_closed)
        }
        popupMenu.show()
        eventText.setText(R.string.popup_opened)
    }

    private fun handlePopupItem(item: MenuItem): Boolean {
        if (item.itemId == R.id.popup_checked_item) {
            popupItemChecked = !item.isChecked
        }
        return when (item.itemId) {
            R.id.popup_one,
            R.id.popup_two,
            R.id.popup_submenu_item,
            R.id.popup_checked_item,
            R.id.popup_six -> {
                resultText.text = getString(R.string.selected_popup, item.
    title)
                true
            }

            else -> false
        }
    }

    private fun showProgrammaticMenu(anchor: View) {
        val popupMenu = PopupMenu(this, anchor)
        popupMenu.menu.add(
            R.id.dynamic_group,
            R.id.dynamic_first,
            1,
            R.string.dynamic_first,
        )
        popupMenu.menu.add(
            R.id.dynamic_group,
            R.id.dynamic_second,
            2,
            R.string.dynamic_second,
        )
        popupMenu.setOnMenuItemClickListener { item ->
            resultText.text = getString(R.string.dynamic_selected, item.
    title)
            true
        }
        popupMenu.setOnDismissListener {
            eventText.setText(R.string.popup_closed)
        }
        popupMenu.show()
        eventText.setText(R.string.popup_opened)
    }
}
```

Ресурс обычного меню menu_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">

    <item
        android:id="@+id/action_settings"
        android:orderInCategory="10"
        android:title="@string/action_settings"
        app:showAsAction="never" />

    <group
        android:id="@+id/language_group"
        android:checkableBehavior="single">

        <item
            android:id="@+id/action_java"
            android:orderInCategory="20"
            android:title="@string/action_java"
            app:showAsAction="never" />

        <item
            android:id="@+id/action_kotlin"
            android:checked="true"
            android:orderInCategory="30"
            android:title="@string/action_kotlin"
            app:showAsAction="never" />

        <item
            android:id="@+id/action_python"
            android:orderInCategory="40"
            android:title="@string/action_python"
            app:showAsAction="never" />
    </group>
</menu>
```

Ресурс всплывающего меню popup_menu.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <group android:id="@+id/popup_group_one">
        <item
            android:id="@+id/popup_one"
            android:icon="@drawable/ic_swift"
            android:title="@string/popup_item_one" />
        <item
            android:id="@+id/popup_two"
            android:title="@string/popup_item_two" />
        <item
            android:id="@+id/popup_three"
            android:title="@string/popup_item_three">
            <menu>
                <item
                    android:id="@+id/popup_submenu_item"
                    android:title="@string/popup_submenu" />
            </menu>
        </item>
    </group>

    <group android:id="@+id/popup_group_two">
        <item
            android:id="@+id/popup_checked_item"
            android:checkable="true"
            android:checked="true"
            android:title="@string/popup_checked" />
        <item
            android:id="@+id/popup_disabled_item"
            android:enabled="false"
            android:title="@string/popup_disabled" />
        <item
            android:id="@+id/popup_six"
            android:title="@string/popup_item_six" />
    </group>
</menu>
```